Variables y expresiones en Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **4**

1 Valores y tipos de datos **4**
1.1 Tipos básicos en Python . . . 4
1.1.1 Integers (Enteros) . . . 4
1.1.2 Floats (Números de punto flotante) . . . 4
1.1.3 Strings (Cadenas de texto) . . . 4
1.1.4 Booleans (Booleanos) . . . 4
1.2 Verificación de tipos . . . 5
1.3 Conversión entre tipos . . . 5
1.4 Ejemplo aplicado: Cálculo del IPC . . . 6

2 Nombres de variables y palabras reservadas **6**
2.1 Reglas para nombres de variables . . . 6
2.2 Palabras reservadas de Python . . . 6
2.3 Convenciones de nombres en Python . . . 7
2.4 Nombres descriptivos vs. cortos . . . 8
2.5 Ejemplo económico completo . . . 8

3 Operadores y operandos **9**
3.1 Operadores aritméticos . . . 9
3.2 Operadores de comparación . . . 9
3.3 Operadores lógicos . . . 10
3.4 Operadores de asignación . . . 11
3.5 Ejemplo económico: Cálculo de interés compuesto . . . 11

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

4 Expresiones **12**
4.1 Expresiones simples . . . 12
4.2 Expresiones complejas . . . 12
4.3 Expresiones en contexto económico . . . 12

5 Orden de operaciones **13**
5.1 Precedencia de operadores . . . 13
5.2 Ejemplos de precedencia . . . 13
5.3 Aplicación económica: Valor presente neto (VPN) . . . 14
5.4 Uso de paréntesis para claridad . . . 14

6 Operaciones con strings **15**
6.1 Concatenación de strings . . . 15
6.2 Repetición de strings . . . 15
6.3 Formato de strings . . . 15
6.4 Métodos de strings . . . 17
6.5 Indexación y slicing de strings . . . 18
6.6 Verificación de contenido . . . 18
6.7 Ejemplo aplicado: Procesamiento de datos económicos . . . 19

7 Solicitando entrada del usuario **19**
7.1 Uso básico de input() . . . 20
7.2 Conversión de entrada a números . . . 20
7.3 Manejo de errores en la entrada . . . 20
7.4 Aplicación económica: Calculadora de interés simple . . . 20
7.5 Ejemplo: Calculadora de elasticidad . . . 21

8 Comentarios **22**
8.1 Comentarios de una línea . . . 22
8.2 Comentarios de múltiples líneas . . . 22
8.3 Buenas prácticas en comentarios . . . 23
8.4 Comentarios para debugging . . . 23

9 Nombres de variables mnemotécnicos **24**
9.1 Principios generales . . . 24
9.2 Ejemplos en contexto económico . . . 25
9.3 Abreviaciones aceptables en economía . . . 25
9.4 Evitar ambigüedades . . . 26

10 Ejercicios aplicados a economía **26**
10.1 Ejercicio 1: Calculadora de índice de pobreza . . . 26
10.2 Ejercicio 2: Cálculo del índice de Gini simplificado . . . 27
10.3 Ejercicio 3: Análisis de punto de equilibrio . . . 28
10.4 Ejercicio 4: Cálculo de la tasa de actividad y desempleo . . . 30
10.5 Ejercicio 5: Conversor de monedas con múltiples divisas . . . 31

11 Conclusión **32**
11.1 Próximos pasos . . . 33
11.2 Recursos adicionales . . . 33

12 Publicaciones Similares **33**

Variables y expresiones en Python

En esta segunda guía exploraremos en profundidad los elementos fundamentales de Python: variables, expresiones y statements. Estos conceptos son la base para cualquier programa que escribamos y son esenciales para el análisis económico con Python.

1 Valores y tipos de datos

1.1 Tipos básicos en Python

Python maneja diferentes tipos de datos que son fundamentales para nuestros análisis económicos. Los tipos básicos son:

1.1.1 Integers (Enteros)

Los integers son números enteros, positivos o negativos, sin decimales. Son útiles para contar unidades discretas como número de trabajadores, cantidad de empresas, años, etc.

```
# Ejemplos de integers en contexto económico
num_trabajadores = 150
año_base = 2010
cantidad_empresas = 45
tasa_desempleo_entera = 7  # 7%
```

1.1.2 Floats (Números de punto flotante)

Los floats son números con decimales. La mayoría de variables económicas son floats: precios, tasas, proporciones, etc.

```
# Ejemplos de floats en economía
precio_dolar = 3.75
tasa_interes = 0.065  # 6.5%
pib_per_capita = 7196.21
inflacion_mensual = 0.42  # 0.42%
```

1.1.3 Strings (Cadenas de texto)

Los strings son secuencias de caracteres. Se usan para nombres, descripciones, códigos, etc.

```
# Ejemplos de strings en economía
nombre_pais = "Perú"
codigo_sector = "CIIU-Rev4"
descripcion = "Producto Bruto Interno a precios constantes"
moneda = "PEN"
```

1.1.4 Booleans (Booleanos)

Los booleans solo pueden tener dos valores: True (verdadero) o False (falso). Son fundamentales para lógica condicional.

```
# Ejemplos de booleans en economía
esta_en_recesion = False
supera_meta_inflacion = True
es_pais_desarrollado = False
tiene_superavit_fiscal = True
```

1.2 Verificación de tipos

Python nos permite verificar el tipo de cualquier valor con la función `type()`:

```
# Verificando tipos
print(type(150))           # <class 'int'>
print(type(3.75))          # <class 'float'>
print(type("Perú"))        # <class 'str'>
print(type(True))          # <class 'bool'>
```

1.3 Conversión entre tipos

Frecuentemente necesitamos convertir datos de un tipo a otro. Python proporciona funciones para esto:

```
# Conversión de float a integer
precio_aproximado = int(3.75)  # Resultado: 3
print(precio_aproximado)

# Conversión de integer a float
tasa_como_float = float(7)  # Resultado: 7.0
print(tasa_como_float)

# Conversión de número a string
año_texto = str(2023)  # Resultado: "2023"
print("El año es: " + año_texto)

# Conversión de string a número
texto_precio = "3.75"
precio_numerico = float(texto_precio)  # Resultado: 3.75
print(precio_numerico)
```

Nota importante sobre conversión de float a int:

```
# La conversión a int trunca, no redondea
print(int(4.9))   # Resultado: 4
print(int(4.1))   # Resultado: 4
print(int(-4.9))  # Resultado: -4

# Para redondear, usamos round()
print(round(4.9))  # Resultado: 5
print(round(4.1))  # Resultado: 4
```

1.4 Ejemplo aplicado: Cálculo del IPC

```
# Datos de precios de canasta básica
precio_canasta_2023 = 850.50
precio_canasta_base = 750.00
año_base = 2020

# Cálculo del IPC
ipc = (precio_canasta_2023 / precio_canasta_base) * 100

# Resultados
print("Cálculo del Índice de Precios al Consumidor")
print("Año base:", año_base)
print("Precio canasta base: S/", precio_canasta_base)
print("Precio canasta actual: S/", precio_canasta_2023)
print("IPC:", round(ipc, 2))
print("Tipo de dato del IPC:", type(ipc))
```

2 Nombres de variables y palabras reservadas

2.1 Reglas para nombres de variables

Python tiene reglas estrictas sobre cómo nombrar variables:
Reglas obligatorias:

1. Debe comenzar con una letra (a-z, A-Z) o guión bajo (_)
2. Puede contener letras, números y guiones bajos
3. No puede contener espacios
4. Es sensible a mayúsculas y minúsculas
5. No puede ser una palabra reservada de Python

```
# Nombres válidos
pib_peru = 242632
tasa_interes_2023 = 6.5
_variable_privada = 100
PIB = 500000  # Diferente de pib

# Nombres inválidos (estos generarán errores)
# 2023_pib = 100        # No puede empezar con número
# tasa-interes = 5.5    # No puede contener guiones
# tasa interes = 5.5    # No puede contener espacios
# class = "Economía"    # class es palabra reservada
```

2.2 Palabras reservadas de Python

Python tiene palabras reservadas que no pueden usarse como nombres de variables:

```
# Lista de palabras reservadas en Python 3
# False      await      else       import     pass
# None       break      except     in         raise
# True       class      finally    is         return
# and        continue   for        lambda     try
# as         def        from       nonlocal   while
# assert     del        global     not        with
# async      elif       if         or         yield
```

2.3 Convenciones de nombres en Python

Python sigue convenciones (no obligatorias pero recomendadas):
Snake case para variables y funciones:

```
# Recomendado: snake_case (palabras separadas por guiones bajos)
precio_producto = 15.50
tasa_crecimiento_pib = 2.4
numero_trabajadores_formales = 8500000

# No recomendado pero válido: camelCase
precioProducto = 15.50
tasaCrecimientoPib = 2.4
```

Constantes en mayúsculas:

```
# Constantes (valores que no cambian)
PI = 3.14159
TASA_IGV = 0.18
SUELDO_MINIMO_VITAL = 1025.00
año_BASE_IPC = 2020
```

Variables privadas con guión bajo:

```
# Primero definimos las variables necesarias
precio_unitario = 25.50  # precio de venta por unidad
cantidad_vendida = 120  # unidades vendidas

# Ahora sí podemos calcular
_calculo_intermedio = precio_unitario * cantidad_vendida
_dato_temporal = 100  # variable que solo usamos internamente

# Mostramos resultados (opcional, para que el lector vea)
print(f"Precio unitario: S/ {precio_unitario:,.2f}")
print(f"Cantidad vendida: {cantidad_vendida} unidades")
print(f"Subtotal (cálculo intermedio): S/ {_calculo_intermedio:,.2f}")
print(f"Valor temporal interno: {_dato_temporal}")
```

2.4 Nombres descriptivos vs. cortos

En programación, es preferible usar nombres descriptivos que hagan el código auto-documentado:

```
# Mal: nombres crípticos
x = 3.75
y = 1000
z = x * y

# Bien: nombres descriptivos
precio_unitario = 3.75
cantidad_vendida = 1000
ingreso_total = precio_unitario * cantidad_vendida

# Para índices en bucles, nombres cortos son aceptables
for i in range(10):
    print(i)
```

2.5 Ejemplo económico completo

```
# Variables macroeconómicas del Perú (valores hipotéticos)
PIB_NOMINAL = 242632  # Constante en millones USD
POBLACION = 33715471  # Constante

# Variables calculadas
pib_per_capita = PIB_NOMINAL / POBLACION * 1000000
tasa_crecimiento_anual = 2.7
pib_proyectado = PIB_NOMINAL * (1 + tasa_crecimiento_anual / 100)

# Información textual
nombre_pais = "Perú"
codigo_iso = "PER"
moneda_local = "Sol peruano"
simbolo_moneda = "S/"

# Variables booleanas
es_economia_emergente = True
pertenece_ocde = False

# Reporte
print(f"Análisis Económico - {nombre_pais} ({codigo_iso})")
print(f"PIB nominal: USD {PIB_NOMINAL:,} millones")
print(f"Población: {POBLACION:,} habitantes")
print(f"PIB per cápita: USD {pib_per_capita:,.2f}")
print(f"Economía emergente: {es_economia_emergente}")
```

3 Operadores y operandos

3.1 Operadores aritméticos

Los operadores son símbolos que indican operaciones. Los operandos son los valores sobre los que se opera.

```
# Suma
ingreso_enero = 5000
ingreso_febrero = 5500
ingreso_total = ingreso_enero + ingreso_febrero
print("Ingreso total:", ingreso_total)  # 10500

# Resta
gastos = 4000
ahorro = ingreso_enero - gastos
print("Ahorro:", ahorro)  # 1000

# Multiplicación
precio_unitario = 25.50
cantidad = 100
valor_venta = precio_unitario * cantidad
print("Valor de venta:", valor_venta)  # 2550.0

# División
pib_total = 500000
numero_habitantes = 50000
pib_per_capita = pib_total / numero_habitantes
print("PIB per cápita:", pib_per_capita)  # 10.0

# Potencia
capital_inicial = 10000
tasa_interes = 0.05
anos = 3
capital_final = capital_inicial * (1 + tasa_interes) ** anos
print("Capital final:", round(capital_final, 2))  # 11576.25

# División entera (cociente)
produccion_total = 1000
tamano_lote = 75
lotes_completos = produccion_total // tamano_lote
print("Lotes completos:", lotes_completos)  # 13

# Módulo (residuo)
unidades_restantes = produccion_total % tamano_lote
print("Unidades restantes:", unidades_restantes)  # 25
```

3.2 Operadores de comparación

Estos operadores comparan valores y devuelven True o False:

```python
# Datos económicos
inflacion_actual = 3.5
meta_inflacion = 3.0
pib_2023 = 250000
pib_2022 = 245000

# Igual a
print(inflacion_actual == meta_inflacion)  # False

# Diferente de
print(inflacion_actual != meta_inflacion)  # True

# Mayor que
print(inflacion_actual > meta_inflacion)  # True

# Menor que
print(inflacion_actual < meta_inflacion)  # False

# Mayor o igual que
print(pib_2023 >= pib_2022)  # True

# Menor o igual que
print(inflacion_actual <= meta_inflacion)  # False
```

3.3 Operadores lógicos

Los operadores lógicos combinan expresiones booleanas:

```python
# Datos para análisis
tasa_desempleo = 7.2
tasa_inflacion = 3.8
crecimiento_pib = 2.5

# AND (y): ambas condiciones deben ser verdaderas
desempleo_bajo = tasa_desempleo < 8.0
inflacion_controlada = tasa_inflacion < 4.0
economia_estable = desempleo_bajo and inflacion_controlada
print("Economía estable:", economia_estable)  # True

# OR (o): al menos una condición debe ser verdadera
hay_problema = tasa_desempleo > 10.0 or tasa_inflacion > 5.0
print("Hay problema económico:", hay_problema)  # False

# NOT (no): invierte el valor booleano
crecimiento_positivo = crecimiento_pib > 0
en_recesion = not crecimiento_positivo
print("En recesión:", en_recesion)  # False
```

3.4 Operadores de asignación

Además del operador básico de asignación (=), Python tiene operadores compuestos:

```
# Asignación básica
capital = 10000

# Asignación con suma
capital += 1500  # Equivalente a: capital = capital + 1500
print("Capital después de aporte:", capital)  # 11500

# Asignación con resta
capital -= 2000  # Equivalente a: capital = capital - 2000
print("Capital después de retiro:", capital)  # 9500

# Asignación con multiplicación
capital *= 1.05  # Equivalente a: capital = capital * 1.05
print("Capital con interés:", capital)  # 9975.0

# Asignación con división
capital /= 2  # Equivalente a: capital = capital / 2
print("Capital dividido:", capital)  # 4987.5

# Asignación con potencia
tasa = 2
tasa **= 3  # Equivalente a: tasa = tasa ** 3
print("Tasa al cubo:", tasa)  # 8
```

3.5 Ejemplo económico: Cálculo de interés compuesto

```
# Parámetros del préstamo
capital_inicial = 50000.00
tasa_anual = 0.08  # 8%
periodo_anos = 5

# Método 1: Usando fórmula directa
capital_final_1 = capital_inicial * (1 + tasa_anual) ** periodo_anos

# Método 2: Usando operador compuesto
capital_final_2 = capital_inicial
for ano in range(periodo_anos):
    capital_final_2 *= (1 + tasa_anual)

# Resultados
print("Préstamo con Interés Compuesto")
print(f"Capital inicial: S/ {capital_inicial:,.2f}")
print(f"Tasa anual: {tasa_anual * 100}%")
print(f"Periodo: {periodo_anos} años")
```

```
print(f"Capital final (método 1): S/ {capital_final_1:,.2f}")
print(f"Capital final (método 2): S/ {capital_final_2:,.2f}")
print(f"Intereses ganados: S/ {capital_final_1 - capital_inicial:,.2f}")
```

4 Expresiones

Una expresión es una combinación de valores, variables y operadores que Python puede evaluar para producir un resultado.

4.1 Expresiones simples

```
# Primero definimos las variables que vamos a usar
precio = 25.50
cantidad = 100

# Expresiones numéricas
5 + 3  # Evalúa a 8
precio * cantidad  # Evalúa al producto
pib_2023 > pib_2022  # Evalúa a True o False

# Expresiones con variables
ingreso_mensual = 3500
gasto_mensual = 2800
ahorro_mensual = ingreso_mensual - gasto_mensual  # Evalúa a 700
```

4.2 Expresiones complejas

```
# Cálculo de la propensión marginal al consumo
consumo_inicial = 50000
consumo_final = 55000
ingreso_inicial = 100000
ingreso_final = 110000

pmc = (consumo_final - consumo_inicial) / (ingreso_final - ingreso_inicial)
print(f"Propensión marginal al consumo: {pmc}")  # 0.5

# Cálculo del multiplicador keynesiano
multiplicador = 1 / (1 - pmc)
print(f"Multiplicador keynesiano: {multiplicador:.2f}")  # 2.00
```

4.3 Expresiones en contexto económico

```
# Cálculo de elasticidad precio de la demanda
precio_inicial = 100
precio_final = 110
cantidad_inicial = 1000
```

```
cantidad_final = 950

# Variación porcentual del precio
variacion_precio = ((precio_final - precio_inicial) / precio_inicial) * 100

# Variación porcentual de la cantidad
variacion_cantidad = ((cantidad_final - cantidad_inicial) / cantidad_inicial) * 100

# Elasticidad
elasticidad = variacion_cantidad / variacion_precio

print(f"Variación del precio: {variacion_precio:.2f}%")
print(f"Variación de la cantidad: {variacion_cantidad:.2f}%")
print(f"Elasticidad precio de la demanda: {elasticidad:.2f}")

# Interpretación
if abs(elasticidad) > 1:
    tipo_demanda = "elástica"
elif abs(elasticidad) < 1:
    tipo_demanda = "inelástica"
else:
    tipo_demanda = "unitaria"

print(f"La demanda es {tipo_demanda}")
```

5 Orden de operaciones

Python sigue un orden específico al evaluar expresiones, similar a las matemáticas tradicionales.

5.1 Precedencia de operadores

De mayor a menor precedencia:

1. Paréntesis: `()`
2. Potencia: `**`
3. Multiplicación, División, División entera, Módulo: `*`, `/`, `//`, `%`
4. Suma, Resta: `+`, `-`
5. Comparación: `<`, `<=`, `>`, `>=`, `==`, `!=`
6. NOT lógico: `not`
7. AND lógico: `and`
8. OR lógico: `or`

5.2 Ejemplos de precedencia

```
# Sin paréntesis
resultado_1 = 2 + 3 * 4
print(resultado_1)  # 14 (primero 3*4=12, luego 2+12=14)
```

```
# Con paréntesis
resultado_2 = (2 + 3) * 4
print(resultado_2)  # 20 (primero 2+3=5, luego 5*4=20)

# Expresión compleja
resultado_3 = 10 + 5 * 2 ** 3
print(resultado_3)  # 50 (2**3=8, 5*8=40, 10+40=50)

# Con paréntesis para claridad
resultado_4 = 10 + (5 * (2 ** 3))
print(resultado_4)  # 50 (mismo resultado pero más claro)
```

5.3 Aplicación económica: Valor presente neto (VPN)

```
# Cálculo del VPN sin usar paréntesis adecuados
inversion_inicial = 100000
flujo_ano1 = 30000
flujo_ano2 = 40000
flujo_ano3 = 50000
tasa_descuento = 0.10

# Incorrecto (orden de operaciones problemático)
# vpn_incorrecto = -inversion_inicial + flujo_ano1 / 1 + tasa_descuento + flujo_ano2

# Correcto (con paréntesis apropiados)
vpn = -inversion_inicial + \
      flujo_ano1 / (1 + tasa_descuento) + \
      flujo_ano2 / (1 + tasa_descuento) ** 2 + \
      flujo_ano3 / (1 + tasa_descuento) ** 3

print(f"Inversión inicial: S/ {inversion_inicial:,.2f}")
print(f"VPN del proyecto: S/ {vpn:,.2f}")

if vpn > 0:
    decision = "Aceptar el proyecto"
else:
    decision = "Rechazar el proyecto"

print(f"Decisión: {decision}")
```

5.4 Uso de paréntesis para claridad

Aunque Python respeta la precedencia, usar paréntesis hace el código más legible:

```
# Menos claro
tasa_crecimiento = consumo_final - consumo_inicial / consumo_inicial * 100
```

```
# Más claro
tasa_crecimiento = ((consumo_final - consumo_inicial) / consumo_inicial) * 100

# Cálculo de la tasa efectiva anual
tasa_nominal = 0.12
periodos = 12
tasa_efectiva = (1 + tasa_nominal / periodos) ** periodos - 1
print(f"Tasa efectiva anual: {tasa_efectiva * 100:.2f}%")
```

6 Operaciones con strings

Los strings son fundamentales para trabajar con datos textuales en economía: nombres de países, descripciones de sectores, códigos, etc.

6.1 Concatenación de strings

```
# Concatenación básica con +
pais = "Perú"
continente = "América del Sur"
descripcion = pais + " está ubicado en " + continente
print(descripcion)

# Concatenación con múltiples variables
nombre = "Edison"
apellido = "Achalma"
profesion = "Economista"
presentacion = nombre + " " + apellido + ", " + profesion
print(presentacion)
```

6.2 Repetición de strings

```
# Repetir caracteres
linea_separadora = "-" * 50
print(linea_separadora)

# Útil para formateo
print("=" * 40)
print("REPORTE ECONÓMICO".center(40))
print("=" * 40)
```

6.3 Formato de strings

Python ofrece varias formas de formatear strings:
Método antiguo con %

```
# Formato con %
pais = "Perú"
pib = 242632
print("El PIB de %s es USD %d millones" % (pais, pib))

# Con floats
inflacion = 3.25
print("La inflación es %.2f%%" % inflacion)

# Múltiples valores
nombre = "Edison"
edad = 30
print("%s tiene %d años" % (nombre, edad))
```

Método format()

```
# Método format básico
pais = "Perú"
pib = 242632
print("El PIB de {} es USD {} millones".format(pais, pib))

# Con índices
print("El PIB de {0} es USD {1} millones. {0} es un país emergente.".format(pais, pib

# Con nombres
print("El PIB de {p} es USD {valor} millones".format(p=pais, valor=pib))

# Con formato numérico
inflacion = 3.254
print("Inflación: {:.2f}%".format(inflacion))  # 2 decimales

# Separador de miles
pib_grande = 242632000000
print("PIB: USD {:,}".format(pib_grande))  # Con comas
```

F-strings (Recomendado, Python 3.6+)

```
# F-strings: forma moderna y más legible
pais = "Perú"
pib = 242632
poblacion = 33715471

# Básico
print(f"El PIB de {pais} es USD {pib} millones")

# Con expresiones
pib_per_capita = pib / poblacion * 1000000
print(f"PIB per cápita: USD {pib_per_capita:.2f}")
```

```
# Con formato
inflacion = 3.254
print(f"Inflación: {inflacion:.2f}%")

# Con separador de miles
print(f"Población: {poblacion:,} habitantes")

# Expresiones complejas dentro del f-string
precio = 100
cantidad = 50
print(f"Valor total: S/ {precio * cantidad:,.2f}")
```

6.4 Métodos de strings

```
# Texto para análisis
sector_economico = "agricultura, ganadería y pesca"

# Convertir a mayúsculas
print(sector_economico.upper())  # AGRICULTURA, GANADERÍA Y PESCA

# Convertir a minúsculas
titulo = "PRODUCTO BRUTO INTERNO"
print(titulo.lower())  # producto bruto interno

# Title case (primera letra de cada palabra en mayúscula)
print(sector_economico.title())  # Agricultura, Ganadería Y Pesca

# Capitalizar (solo primera letra en mayúscula)
print(sector_economico.capitalize())  # Agricultura, ganadería y pesca

# Reemplazar texto
texto_original = "La tasa de interés es 5%"
texto_modificado = texto_original.replace("5%", "6%")
print(texto_modificado)  # La tasa de interés es 6%

# Dividir strings
sectores = "Minería, Agricultura, Manufactura, Servicios"
lista_sectores = sectores.split(", ")
print(lista_sectores)  # ['Minería', 'Agricultura', 'Manufactura', 'Servicios']

# Unir strings
palabras = ["Producto", "Bruto", "Interno"]
frase = " ".join(palabras)
print(frase)  # Producto Bruto Interno

# Eliminar espacios al inicio y final
```

```
texto_con_espacios = "   PIB   "
texto_limpio = texto_con_espacios.strip()
print(f"'{texto_limpio}'")  # 'PIB'
```

6.5 Indexación y slicing de strings

Los strings son secuencias de caracteres que pueden ser indexados:

```
# String de ejemplo
indicador = "PRODUCTO BRUTO INTERNO"

# Indexación (posiciones empiezan en 0)
print(indicador[0])    # P (primer carácter)
print(indicador[1])    # R (segundo carácter)
print(indicador[-1])   # O (último carácter)
print(indicador[-2])   # N (penúltimo carácter)

# Slicing (obtener subsecuencias)
print(indicador[0:8])   # PRODUCTO (desde 0 hasta 8, no incluye 8)
print(indicador[:8])    # PRODUCTO (desde inicio hasta 8)
print(indicador[9:14])  # BRUTO
print(indicador[15:])   # INTERNO (desde 15 hasta el final)
print(indicador[:])     # PRODUCTO BRUTO INTERNO (copia completa)

# Slicing con paso
print(indicador[::2])   # POUOBUTITRO (cada 2 caracteres)
print(indicador[::-1])  # ONRETNI OTURB OTCUDORP (invertido)
```

6.6 Verificación de contenido

```
# Datos económicos
codigo_sector = "CIIU-456"
descripcion = "Comercio al por menor"

# Verificar si contiene
print("CIIU" in codigo_sector)  # True
print("manufactura" in descripcion.lower())  # False

# Verificar inicio
print(codigo_sector.startswith("CIIU"))  # True
print(descripcion.startswith("Comercio"))  # True

# Verificar final
print(codigo_sector.endswith("456"))  # True
print(descripcion.endswith("mayor"))  # True

# Verificar si es numérico
```

```
año = "2023"
print(año.isdigit())  # True

precio = "123.45"
print(precio.isdigit())  # False (por el punto decimal)

# Verificar si es alfabético
moneda = "soles"
print(moneda.isalpha())  # True
```

6.7 Ejemplo aplicado: Procesamiento de datos económicos

```
# Datos de sectores económicos (simulando datos crudos)
datos_brutos = "minería;45.2;agricultura;12.8;manufactura;23.5;servicios;67.9"

# Separar datos
lista_datos = datos_brutos.split(";")

# Procesar en pares (sector, valor)
sectores = []
valores = []

for i in range(0, len(lista_datos), 2):
    sector = lista_datos[i].title()
    valor = float(lista_datos[i + 1])
    sectores.append(sector)
    valores.append(valor)

# Generar reporte
print("CONTRIBUCIÓN SECTORIAL AL PIB")
print("=" * 40)

for sector, valor in zip(sectores, valores):
    barra = " " * int(valor / 2)  # Gráfico simple
    print(f"{sector:15} {valor:6.1f}% {barra}")

# Total
total = sum(valores)
print("=" * 40)
print(f"{'TOTAL':15} {total:6.1f}%")
```

7 Solicitando entrada del usuario

La función `input()` permite interactuar con el usuario, solicitando datos durante la ejecución del programa.

7.1 Uso básico de input()

```
# Solicitar nombre
nombre = input("Ingrese su nombre: ")
print(f"Hola, {nombre}!")

# Solicitar país
pais = input("¿De qué país eres? ")
print(f"Interesante, {pais} es un gran país.")
```

7.2 Conversión de entrada a números

Importante: `input()` siempre devuelve un string. Para trabajar con números, debemos convertir:

```
# Solicitar edad (necesita conversión a int)
edad_texto = input("Ingrese su edad: ")
edad = int(edad_texto)
print(f"Usted tiene {edad} años")

# Forma abreviada
edad = int(input("Ingrese su edad: "))

# Solicitar valor decimal
precio = float(input("Ingrese el precio del producto: "))
cantidad = int(input("Ingrese la cantidad: "))
total = precio * cantidad
print(f"Total a pagar: S/ {total:.2f}")
```

7.3 Manejo de errores en la entrada

```
# Si el usuario ingresa texto cuando esperamos número
try:
    edad = int(input("Ingrese su edad: "))
    print(f"Su edad es {edad}")
except ValueError:
    print("Error: Debe ingresar un número válido")
```

7.4 Aplicación económica: Calculadora de interés simple

```
# Calculadora interactiva de interés simple
print("=" * 50)
print("CALCULADORA DE INTERÉS SIMPLE")
print("=" * 50)

# Solicitar datos al usuario
```

```
capital = float(input("Ingrese el capital inicial (S/): "))
tasa = float(input("Ingrese la tasa de interés anual (%): "))
tiempo = int(input("Ingrese el tiempo en años: "))

# Cálculos
tasa_decimal = tasa / 100
interes = capital * tasa_decimal * tiempo
monto_final = capital + interes

# Resultados
print("\n" + "=" * 50)
print("RESULTADOS")
print("=" * 50)
print(f"Capital inicial:  S/ {capital:,.2f}")
print(f"Tasa de interés:  {tasa}%")
print(f"Tiempo:           {tiempo} años")
print(f"Interés ganado:   S/ {interes:,.2f}")
print(f"Monto final:      S/ {monto_final:,.2f}")
print("=" * 50)
```

7.5 Ejemplo: Calculadora de elasticidad

```
# Calculadora de elasticidad precio de la demanda
print("CALCULADORA DE ELASTICIDAD PRECIO DE LA DEMANDA")
print("-" * 50)

# Datos iniciales
print("\nDatos iniciales:")
precio_inicial = float(input("Precio inicial: "))
cantidad_inicial = float(input("Cantidad inicial: "))

# Datos finales
print("\nDatos finales:")
precio_final = float(input("Precio final: "))
cantidad_final = float(input("Cantidad final: "))

# Cálculos
var_precio = ((precio_final - precio_inicial) / precio_inicial) * 100
var_cantidad = ((cantidad_final - cantidad_inicial) / cantidad_inicial) * 100
elasticidad = var_cantidad / var_precio

# Resultados
print("\n" + "=" * 50)
print("RESULTADOS")
print("=" * 50)
print(f"Variación del precio:    {var_precio:+.2f}%")
print(f"Variación de la cantidad: {var_cantidad:+.2f}%")
```

```
print(f"Elasticidad:              {elasticidad:.2f}")

# Interpretación
print("\nInterpretación:")
if abs(elasticidad) > 1:
    print("La demanda es ELÁSTICA")
    print("Los consumidores son sensibles a cambios en el precio")
elif abs(elasticidad) < 1:
    print("La demanda es INELÁSTICA")
    print("Los consumidores son poco sensibles a cambios en el precio")
else:
    print("La demanda es UNITARIA")
    print("La sensibilidad es proporcional")
```

8 Comentarios

Los comentarios son líneas de código que Python ignora. Sirven para documentar y explicar el código.

8.1 Comentarios de una línea

```
# Este es un comentario de una línea
# Python ignora esta línea

# Cálculo del PIB per cápita
pib_total = 242632  # PIB en millones de USD
poblacion = 33715471  # Población total

# Realizar el cálculo
pib_per_capita = (pib_total / poblacion) * 1000000  # Convertir a USD
```

8.2 Comentarios de múltiples líneas

```
"""
Este es un comentario de múltiples líneas.
Útil para documentación más extensa.
Se puede usar comillas triples simples o dobles.
"""

'''
Análisis de Indicadores Macroeconómicos
Autor: Edison Achalma
Fecha: Enero 2026
Descripción: Este script calcula los principales
indicadores macroeconómicos del Perú.
'''
```

```python
def calcular_pib_per_capita(pib, poblacion):
    """
    Calcula el PIB per cápita.

    Parámetros:
    pib: PIB total en millones de USD
    poblacion: Población total

    Retorna:
    PIB per cápita en USD
    """
    return (pib / poblacion) * 1000000
```

8.3 Buenas prácticas en comentarios

```python
# Mal: comentario obvio
x = 5  # asignar 5 a x

# Bien: comentario que explica el propósito
tasa_igv = 0.18  # Tasa de IGV vigente en Perú

# Mal: comentario redundante
precio = precio + 10  # sumar 10 al precio

# Bien: comentario que explica la lógica de negocio
precio += 10  # Ajuste por inflación mensual

# Comentarios para secciones
# ============================================================
# CÁLCULOS MACROECONÓMICOS
# ============================================================

# Variables de entrada
pib_nominal = 242632
deflactor = 1.15

# Cálculo del PIB real
pib_real = pib_nominal / deflactor

# ============================================================
# GENERACIÓN DE REPORTES
# ============================================================
```

8.4 Comentarios para debugging

```python
# Cálculo de elasticidad
precio_inicial = 100
precio_final = 110
cantidad_inicial = 1000
cantidad_final = 950

# print("Debug - precio_inicial:", precio_inicial)  # Comentado temporalmente
# print("Debug - cantidad_inicial:", cantidad_inicial)

elasticidad = ((cantidad_final - cantidad_inicial) / cantidad_inicial) / \
              ((precio_final - precio_inicial) / precio_inicial)

print(f"Elasticidad: {elasticidad:.2f}")
```

9 Nombres de variables mnemotécnicos

Los nombres mnemotécnicos son nombres de variables que ayudan a recordar su propósito sin necesidad de comentarios.

9.1 Principios generales

Usa nombres descriptivos

```python
# Mal
x = 242632
y = 33715471
z = x / y * 1000000

# Bien
pib_total_millones = 242632
poblacion_total = 33715471
pib_per_capita = pib_total_millones / poblacion_total * 1000000
```

Usa convenciones consistentes

```python
# Consistente para tasas
tasa_interes_activa = 0.12
tasa_interes_pasiva = 0.05
tasa_inflacion_anual = 0.035

# Consistente para precios
precio_producto_a = 25.50
precio_producto_b = 30.00
precio_promedio = (precio_producto_a + precio_producto_b) / 2
```

Usa prefijos y sufijos significativos

```
# Prefijos temporales
pib_2022 = 238000
pib_2023 = 242632
pib_2024_proyectado = 250000

# Sufijos de unidades
ingreso_mensual_soles = 3500
ingreso_anual_soles = ingreso_mensual_soles * 12
ingreso_anual_usd = ingreso_anual_soles / 3.75

# Sufijos de tipo
tasa_interes_nominal = 0.12
tasa_interes_real = 0.087
tasa_interes_efectiva = 0.1268
```

9.2 Ejemplos en contexto económico

```
# Variables macroeconómicas
pib_nominal_millones_usd = 242632
pib_real_millones_usd = 210897
deflactor_pib = pib_nominal_millones_usd / pib_real_millones_usd

# Variables de empleo
poblacion_economicamente_activa = 18500000
poblacion_ocupada = 17200000
poblacion_desocupada = poblacion_economicamente_activa - poblacion_ocupada
tasa_desempleo_porcentaje = (poblacion_desocupada / poblacion_economicamente_activa)

# Variables de comercio exterior
exportaciones_millones_usd = 63200
importaciones_millones_usd = 53800
balanza_comercial_millones_usd = exportaciones_millones_usd - importaciones_millones_
es_superavit_comercial = balanza_comercial_millones_usd > 0

# Variables financieras
capital_inicial_soles = 10000
tasa_interes_anual_decimal = 0.08
numero_periodos_anos = 5
capital_final_soles = capital_inicial_soles * (1 + tasa_interes_anual_decimal) ** num
```

9.3 Abreviaciones aceptables en economía

```
# Abreviaciones comunes y aceptadas
pib = 242632  # Producto Bruto Interno
pnb = 245000  # Producto Nacional Bruto
ipc = 113.5   # Índice de Precios al Consumidor
```

```
inpc = 115.2  # Índice Nacional de Precios al Consumidor
igv = 0.18    # Impuesto General a las Ventas
ir = 0.295    # Impuesto a la Renta
var = 12.5    # Valor Agregado Rentable
vpn = 25000   # Valor Presente Neto
tir = 0.15    # Tasa Interna de Retorno

# Cuando usar abreviaciones
# Bien: indicadores conocidos
var_pib = ((pib_2023 - pib_2022) / pib_2022) * 100

# Bien: dentro de contexto claro
def calcular_vpn(flujos, tasa):
    """Calcula el Valor Presente Neto"""
    vpn = sum(flujo / (1 + tasa) ** t for t, flujo in enumerate(flujos))
    return vpn
```

9.4 Evitar ambigüedades

```
# Ambiguo
tasa = 0.05  # ¿Qué tipo de tasa?

# Claro
tasa_impuesto_renta = 0.295
tasa_interes_prestamo = 0.12
tasa_crecimiento_pib = 0.027

# Ambiguo
precio = 100  # ¿En qué moneda? ¿Qué producto?

# Claro
precio_petroleo_barril_usd = 75.50
precio_pan_unidad_soles = 0.50
precio_dolar_interbancario_soles = 3.75
```

10 Ejercicios aplicados a economía

10.1 Ejercicio 1: Calculadora de índice de pobreza

Crea un programa que calcule el índice de pobreza dado el ingreso per cápita y la línea de pobreza.

```
# Ingreso per cápita mensual del hogar
ingreso_per_capita = float(input("Ingrese el ingreso per cápita mensual (S/): "))

# Línea de pobreza (valor oficial)
linea_pobreza_monetaria = 378.00  # S/ mensuales (valor hipotético)
```

```
linea_pobreza_extrema = 201.00  # S/ mensuales (valor hipotético)

# Clasificación
if ingreso_per_capita < linea_pobreza_extrema:
    clasificacion = "Pobreza extrema"
elif ingreso_per_capita < linea_pobreza_monetaria:
    clasificacion = "Pobreza no extrema"
else:
    clasificacion = "No pobre"

# Calcular brecha
if ingreso_per_capita < linea_pobreza_monetaria:
    brecha = linea_pobreza_monetaria - ingreso_per_capita
    porcentaje_brecha = (brecha / linea_pobreza_monetaria) * 100
else:
    brecha = 0
    porcentaje_brecha = 0

# Resultados
print("\n" + "=" * 50)
print("ANÁLISIS DE POBREZA MONETARIA")
print("=" * 50)
print(f"Ingreso per cápita:       S/ {ingreso_per_capita:.2f}")
print(f"Línea de pobreza:         S/ {linea_pobreza_monetaria:.2f}")
print(f"Clasificación:            {clasificacion}")
if brecha > 0:
    print(f"Brecha de pobreza:        S/ {brecha:.2f}")
    print(f"Porcentaje de brecha:     {porcentaje_brecha:.1f}%")
```

10.2 Ejercicio 2: Cálculo del índice de Gini simplificado

```
# Datos de ingreso de 5 hogares (del más pobre al más rico)
print("Ingrese los ingresos de 5 hogares (de menor a mayor):")
ingreso_1 = float(input("Hogar 1 (más pobre): "))
ingreso_2 = float(input("Hogar 2: "))
ingreso_3 = float(input("Hogar 3: "))
ingreso_4 = float(input("Hogar 4: "))
ingreso_5 = float(input("Hogar 5 (más rico): "))

# Total de ingresos
total_ingresos = ingreso_1 + ingreso_2 + ingreso_3 + ingreso_4 + ingreso_5

# Proporción acumulada de ingresos
prop_1 = ingreso_1 / total_ingresos
prop_2 = (ingreso_1 + ingreso_2) / total_ingresos
prop_3 = (ingreso_1 + ingreso_2 + ingreso_3) / total_ingresos
prop_4 = (ingreso_1 + ingreso_2 + ingreso_3 + ingreso_4) / total_ingresos
```

```
prop_5 = 1.0

# Cálculo simplificado del Gini (método trapezoidal)
area_bajo_lorenz = (
    (0.2 * prop_1)
    + (0.2 * (prop_1 + prop_2)) / 2
    + (0.2 * (prop_2 + prop_3)) / 2
    + (0.2 * (prop_3 + prop_4)) / 2
    + (0.2 * (prop_4 + prop_5)) / 2
)

gini = (0.5 - area_bajo_lorenz) / 0.5

# Resultados
print("\n" + "=" * 50)
print("ANÁLISIS DE DESIGUALDAD - ÍNDICE DE GINI")
print("=" * 50)
print(f"Total de ingresos: S/ {total_ingresos:,.2f}")
print(f"\nDistribución acumulada:")
print(f"20% más pobre:  {prop_1 * 100:.1f}% del ingreso")
print(f"40% más pobre:  {prop_2 * 100:.1f}% del ingreso")
print(f"60% más pobre:  {prop_3 * 100:.1f}% del ingreso")
print(f"80% más pobre:  {prop_4 * 100:.1f}% del ingreso")
print(f"\nÍndice de Gini: {gini:.3f}")

# Interpretación
if gini < 0.3:
    interpretacion = "Baja desigualdad"
elif gini < 0.5:
    interpretacion = "Desigualdad moderada"
else:
    interpretacion = "Alta desigualdad"

print(f"Interpretación: {interpretacion}")
```

10.3 Ejercicio 3: Análisis de punto de equilibrio

```
# Análisis de punto de equilibrio para una empresa
print("ANÁLISIS DE PUNTO DE EQUILIBRIO")
print("=" * 50)

# Datos de costos
costos_fijos = float(input("Costos fijos mensuales (S/): "))
costo_variable_unitario = float(input("Costo variable por unidad (S/): "))
precio_venta_unitario = float(input("Precio de venta por unidad (S/): "))

# Validación
```

```
if precio_venta_unitario <= costo_variable_unitario:
    print("\nError: El precio de venta debe ser mayor al costo variable.")
else:
    # Cálculo del punto de equilibrio
    margen_contribucion_unitario = precio_venta_unitario - costo_variable_unitario
    margen_contribucion_porcentaje = (
        margen_contribucion_unitario / precio_venta_unitario
    ) * 100

    punto_equilibrio_unidades = costos_fijos / margen_contribucion_unitario
    punto_equilibrio_soles = punto_equilibrio_unidades * precio_venta_unitario

    # Resultados
    print("\n" + "=" * 50)
    print("RESULTADOS DEL ANÁLISIS")
    print("=" * 50)
    print(f"Costos fijos:                S/ {costos_fijos:,.2f}")
    print(f"Costo variable unitario:     S/ {costo_variable_unitario:,.2f}")
    print(f"Precio de venta unitario:    S/ {precio_venta_unitario:,.2f}")
    print(f"Margen de contribución:      S/ {margen_contribucion_unitario:,.2f}")
    print(f"Margen de contribución:      {margen_contribucion_porcentaje:.1f}%")
    print("\n" + "-" * 50)
    print(f"Punto de equilibrio:         {punto_equilibrio_unidades:.0f} unidades")
    print(f"Punto de equilibrio:         S/ {punto_equilibrio_soles:,.2f}")
    print("=" * 50)

    # Análisis adicional
    print("\nAnálisis de sensibilidad:")
    unidades_vender = float(input("¿Cuántas unidades espera vender? "))

    if unidades_vender > punto_equilibrio_unidades:
        diferencia = unidades_vender - punto_equilibrio_unidades
        utilidad = diferencia * margen_contribucion_unitario
        print(f"Utilidad esperada: S/ {utilidad:,.2f}")
        print("La empresa obtendría ganancias.")
    elif unidades_vender < punto_equilibrio_unidades:
        diferencia = punto_equilibrio_unidades - unidades_vender
        perdida = diferencia * margen_contribucion_unitario
        print(f"Pérdida esperada: S/ {perdida:,.2f}")
        print("La empresa tendría pérdidas.")
    else:
        print(
            "La empresa estaría en el punto de equilibrio (sin ganancias ni pérdidas)
        )
```

10.4 Ejercicio 4: Cálculo de la tasa de actividad y desempleo

```
# Indicadores del mercado laboral
print("CÁLCULO DE INDICADORES DEL MERCADO LABORAL")
print("=" * 50)

# Datos de población
poblacion_total = int(input("Población total: "))
poblacion_menor_14 = int(input("Población menor de 14 años: "))
poblacion_ocupada = int(input("Población ocupada: "))
poblacion_desocupada = int(input("Población desocupada: "))

# Cálculos
poblacion_14_mas = poblacion_total - poblacion_menor_14
poblacion_economicamente_activa = poblacion_ocupada + poblacion_desocupada
poblacion_economicamente_inactiva = poblacion_14_mas - poblacion_economicamente_activ

# Tasas
tasa_actividad = (poblacion_economicamente_activa / poblacion_14_mas) * 100
tasa_desempleo = (poblacion_desocupada / poblacion_economicamente_activa) * 100
tasa_ocupacion = (poblacion_ocupada / poblacion_14_mas) * 100
tasa_inactividad = (poblacion_economicamente_inactiva / poblacion_14_mas) * 100

# Resultados
print("\n" + "=" * 50)
print("INDICADORES DEL MERCADO LABORAL")
print("=" * 50)
print(f"Población total:              {poblacion_total:,}")
print(f"Población de 14 años y más:   {poblacion_14_mas:,}")
print(f"PEA (activa):                 {poblacion_economicamente_activa:,}")
print(f"  - Ocupada:                  {poblacion_ocupada:,}")
print(f"  - Desocupada:               {poblacion_desocupada:,}")
print(f"PEI (inactiva):               {poblacion_economicamente_inactiva:,}")
print("\n" + "-" * 50)
print("TASAS")
print("-" * 50)
print(f"Tasa de actividad:            {tasa_actividad:.2f}%")
print(f"Tasa de desempleo:            {tasa_desempleo:.2f}%")
print(f"Tasa de ocupación:            {tasa_ocupacion:.2f}%")
print(f"Tasa de inactividad:          {tasa_inactividad:.2f}%")
print("=" * 50)

# Análisis
print("\nAnálisis:")
if tasa_desempleo < 5:
    print("Situación de pleno empleo (desempleo friccional)")
elif tasa_desempleo < 10:
    print("Nivel de desempleo moderado")
```

```
else:
    print("Nivel de desempleo elevado")

if tasa_actividad > 60:
    print("Alta participación laboral")
else:
    print("Baja participación laboral")
```

10.5 Ejercicio 5: Conversor de monedas con múltiples divisas

```
# Conversor de monedas
print("CONVERSOR DE MONEDAS")
print("=" * 50)

# Tipos de cambio (respecto al sol peruano)
TC_USD = 3.75  # 1 USD = 3.75 PEN
TC_EUR = 4.10  # 1 EUR = 4.10 PEN
TC_BRL = 0.75  # 1 BRL = 0.75 PEN
TC_CLP = 0.0042  # 1 CLP = 0.0042 PEN

# Menú
print("\nMonedas disponibles:")
print("1. Soles (PEN)")
print("2. Dólares americanos (USD)")
print("3. Euros (EUR)")
print("4. Reales brasileños (BRL)")
print("5. Pesos chilenos (CLP)")

# Selección de monedas
moneda_origen = int(input("\nSeleccione la moneda de origen (1-5): "))
moneda_destino = int(input("Seleccione la moneda de destino (1-5): "))
monto = float(input("Ingrese el monto a convertir: "))

# Convertir a soles primero
if moneda_origen == 1:
    monto_en_soles = monto
elif moneda_origen == 2:
    monto_en_soles = monto * TC_USD
elif moneda_origen == 3:
    monto_en_soles = monto * TC_EUR
elif moneda_origen == 4:
    monto_en_soles = monto * TC_BRL
elif moneda_origen == 5:
    monto_en_soles = monto * TC_CLP
else:
    print("Moneda de origen inválida")
    monto_en_soles = 0
```

```
# Convertir de soles a moneda destino
if monto_en_soles > 0:
    if moneda_destino == 1:
        monto_final = monto_en_soles
        simbolo_destino = "S/"
    elif moneda_destino == 2:
        monto_final = monto_en_soles / TC_USD
        simbolo_destino = "USD"
    elif moneda_destino == 3:
        monto_final = monto_en_soles / TC_EUR
        simbolo_destino = "EUR"
    elif moneda_destino == 4:
        monto_final = monto_en_soles / TC_BRL
        simbolo_destino = "BRL"
    elif moneda_destino == 5:
        monto_final = monto_en_soles / TC_CLP
        simbolo_destino = "CLP"
    else:
        print("Moneda de destino inválida")
        monto_final = 0
        simbolo_destino = ""

    # Resultados
    if monto_final > 0:
        print("\n" + "=" * 50)
        print("RESULTADO DE LA CONVERSIÓN")
        print("=" * 50)
        print(f"Monto convertido: {simbolo_destino} {monto_final:,.2f}")
        print("=" * 50)
```

11 Conclusión

En esta guía hemos cubierto los fundamentos de Python para análisis económico:

Tipos de datos básicos

Los cuatro tipos fundamentales son integers, floats, strings y booleans. Cada uno tiene usos específicos en análisis económico.

Variables y nombres

Las variables deben tener nombres descriptivos que sigan las convenciones de Python (snake_case). Los nombres mnemotécnicos mejoran la legibilidad del código.

Operadores

Python proporciona operadores aritméticos, de comparación y lógicos que son fundamentales para cálculos económicos.

Expresiones

Las expresiones combinan valores y operadores. Python sigue un orden de precedencia específico que puede controlarse con paréntesis.

Strings

Los strings son esenciales para trabajar con datos textuales. Python ofrece múltiples métodos para manipularlos y formatearlos.

Input del usuario

La función `input()` permite crear programas interactivos. Es importante recordar convertir la entrada a números cuando sea necesario.

Comentarios

Los comentarios documentan el código. Son esenciales para mantener código legible y mantenible.

11.1 Próximos pasos

En la siguiente guía (Guía 3: Objetos de Python) exploraremos estructuras de datos más complejas:

- Listas: colecciones ordenadas de elementos
- Diccionarios: pares clave-valor para datos estructurados
- Tuplas: colecciones inmutables
- Operaciones con estas estructuras aplicadas a economía

11.2 Recursos adicionales

Para profundizar en estos conceptos, se recomienda:

- Documentación oficial de Python: docs.python.org
- Práctica constante con ejercicios económicos
- Exploración de bibliotecas como NumPy y Pandas

---

tipo: fragmento titulo: texto de apertura del bloque «Publicaciones similares» (incluido en cada post) —

12 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. 11 Validacion Cruzada Y Composicion Del Modelo Con Python
14. Visualizacion De Datos Con Python

---

tipo: fragmento titulo: texto de cierre del bloque «Publicaciones similares» (incluido en cada post) —

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!